

Лабораторная работа 8

РАБОТА С ДАННЫМИ В ФОРМАТЕ XML

Цель работы: получить практические навыки обработки данных, представленных в формате Extensible Markup Language (XML).

Задачи работы:

- изучить технологию описания данных XML;
- изучить основные классы библиотеки FCL, поддерживающие обработку данных, представленных в формате XML;
- выполнить практическое задание по разработке приложения на языке C#.

Краткие теоретические сведения

Формат XML – это способ хранения данных, представленных в простом текстовом формате, что означает, что эти данные могут быть прочитаны практически любым компьютером. Данное обстоятельство делает этот формат весьма подходящим для использования при передаче данных через Интернет и допускает даже непосредственное прочтение человеком.

XML является языком разметки, с помощью которого можно описать произвольные данные. На основе этого языка можно организовать хранение информации и ее обмен, не зависящий ни от конкретных приложений, ни от платформы, на которой они исполняются.

XML-документы. Законченный набор данных известен в языке XML под названием XML-документа. XML-документ может представлять собой физический файл на вашем компьютере, а может быть всего лишь строкой в памяти, однако он должен быть законченным и подчиняться определенным правилам. XML-документ состоит из нескольких различных частей, наиболее важными из которых являются XML-элементы, где содержатся те данные, из которых собственно и состоит документ.

Microsoft .NET Framework использует объектную модель данных XML Document Object Model (DOM), чтобы обеспечить доступ к данным в XML-документах, и дополнительные классы для чтения, записи и навигации в пределах XML-документа. Эти классы поддерживаются пространством имен **System.XML**. Пример представления описания каталога книг в модели DOM приведен на рис. 8.1.

Описание документа на языке XML включает в себя операторы, написанные с соблюдением требований его синтаксиса. При создании XML-документа вместо использования ограниченного набора определенных элементов имеется возможность создавать собственные элементы и присваивать им любые имена по выбору пользователя. Именно поэтому язык XML является расширяемым (extensible). Следовательно, этот язык можно использовать для описания практически любого документа: от музыкальной партитуры до базы данных.

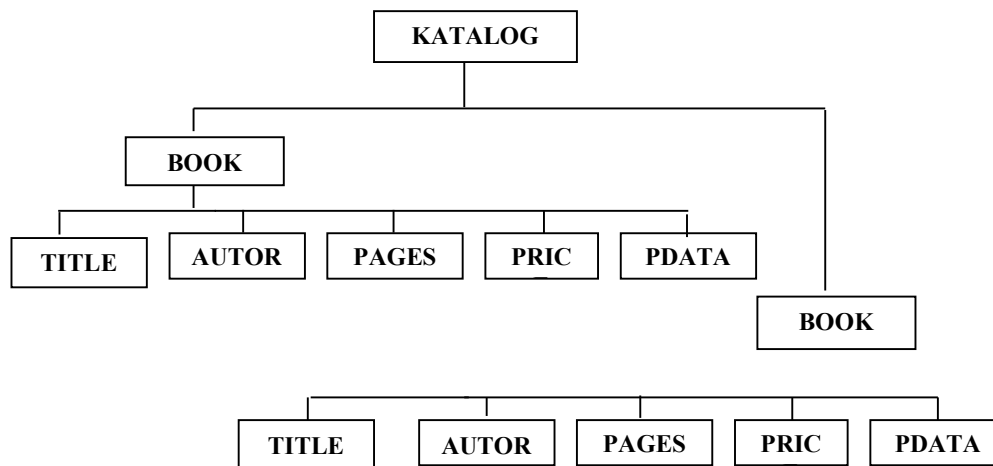


Рис. 8.1. Иерархическая структура документа

Например, каталог книг можно описать так, как показано в листинге 8.1 (номера строк не являются частью документа XML). Для создания XML-документа в среде Visual Studio .NET следует воспользоваться командой File\New File и в выпавшем списке шаблонов выбрать имя шаблона XML File.

Листинг 8.1. Текст XML-документа

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <!-- Название файла book.xml-->
3. <KATALOG>
4. <BOOK>
5. <TITLE>С#2005 для профессионалов </TITLE>
6. <AUTHOR>Кристиан Нейгел</AUTHOR>
7. <PAGES>1560</PAGES>
8. <PRICE>800</PRICE>
9. <PDATA>2006 </PDATA>
10. </BOOK>
11. <BOOK>
12. <TITLE>С# в задачах и примерах</TITLE>
  
```

```
13.<AUTHOR>Никита Культи</AUTHOR>
14.<PAGES>240</PAGES>
15.<PRICE>350</PRICE>
16.<PDATA>2007 </PDATA>
17.</BOOK>
18.</KATALOG>
```

В строке 1 данного листинга записано объявление XML, идентифицирующее текст как документ XML. Несмотря на необязательность объявления XML, документ должен включать его в себя для идентификации используемой версии XML, поскольку документ без объявления XML может в дальнейшем рассматриваться как соответствующий последней версии XML, в результате чего могут появиться ошибки. Информационный параметр **version** указывает версию XML, использованную в документе, параметр **encoding** – кодировку документа (**utf-8**).

В строке 2 записан комментарий, начинающийся с символов **<!--** и заканчивающийся символами **-->**. Комментарии можно размещать по всему XML-документу.

В XML-документе данные маркируются с помощью *тэгов* (элементов), представляющих собой имена, заключенные в угловые скобки (**< >**). Имена тэгов в XML-документе (такие как **KATALOG**, **BOOK**, **TITLE**, **AUTHOR**, **PAGES**, **PRICE**, **PDATA** в листинге 8.1) не являются определениями языка XML и назначаются при создании документа. Для тэгов можно выбирать любые корректно заданные имена, например **INVENTORY** вместо **KATALOG** либо **ITEM** вместо **BOOK**. В строке 3 записан корневой тэг – **KATALOG**, открывающий разметку всего документа. При завершении написания корневого тэга среда автоматически вставляет конечный тэг (строка 18 листинга 8.1), отмечая его символами **</**.

Примечание. Попытка создания более одного корневого элемента в XML-документе является ошибкой.

Внутри корневого элемента может находиться произвольное количество вложенных элементов. В листинге 8.1 XML-документ имеет иерархическую структуру в виде дерева с элементами, вложенными в другие элементы, и с одним элементом верхнего уровня элемент Документ, или Корневой элемент (в нашем примере – **KATALOG**), который содержит все другие элементы. Корневой элемент **KATALOG** включает в себя элементы-потомки **BOOK**. В свою очередь элемент **BOOK** состоит из элементов-потомков **TITLE**, **AUTHOR**, **PAGES**, **PRICE**, **PDATA**.

Корректно сформированные XML-документы. Документ называется *корректно сформированным* (well-formed), если он соответствует следующему минимальному набору правил для XML-документов:

- XML-документ должен иметь только один корневой элемент – элемент **Документ**. Все другие элементы должны быть вложены в корневой элемент;

- элементы должны быть вложены упорядоченным образом. Если элемент начинается внутри другого элемента, то он должен и заканчиваться внутри этого элемента;

- каждый элемент должен иметь начальный и конечный тэги. В отличие от языка HTML, в языке XML не разрешается опускать конечный тэг даже в том случае, когда браузер в состоянии определить, где заканчивается элемент;

- название элемента в начальном тэге должно точно соответствовать (с учетом регистра) названию в соответствующем конечном тэге;

- название элемента должно начинаться с буквы или с символа подчеркивания (_), после чего могут идти буквы, цифры, а также символы: точка (.), тире (-) или подчеркивание.

Это базовые правила корректного формирования XML-документа. Для других понятий языка XML (атрибутов, примитивов, связей) действуют свои правила, которые необходимо соблюдать. Можно сказать, что если документ создан правильно и при его отображении и использовании не возникает никаких ошибок, то это и есть корректно сформированный документ. Если вы ошибетесь в каком-либо тэге HTML-страницы, то браузер просто проигнорирует соответствующий тэг, а ошибка в тэге XML-страницы сделает невозможным ее отображение. При наличии одной из ошибок встроенный в Internet Explorer анализатор (его иногда называют XML-про-цессором, или парсером) определяет ее позицию

Классы библиотеки FCL для чтения XML-файлов. Работу с XML-документами поддерживают следующие классы библиотеки FCL: **XmlTextReader**, **XmlDocument**, **XPathNavigator**.

Класс **XmlTextReader** – это абстрактный класс, выполняющий чтение и обеспечивающий быструю доставку неэкшированных данных. Этот подход в отношении серверных ресурсов является наименее дорогостоящим, но он принуждает извлекать данные последовательно, от начала до конца.

Класс **XmlDocument** представляет собой реализацию модели **DOM**. Этот класс удерживает данные в памяти после вызова метода **Load()** для извлечения их из файла или потока, обеспечивает

древовидное представление документа в памяти с возможностями навигации и редактирования, а также позволяет модифицировать данные и сохранять их обратно в файл.

Класс **XPathNavigator** так же, как и класс **XmlDocument**, удерживает в памяти **XML**-документ целиком. Он предоставляет расширенные средства поиска данных, однако не обеспечивает возможности внесения изменений и их сохранения.

Класс XmlTextReader. Рассмотрим простой пример. Разместим на форме элементы **richTextBox** и **button** (рис. 8.2). При щелчке на кнопку в элемент **richTextBox** будет загружаться файл, содержимое которого было представлено в листинге 8.1. Код функции, вызываемой при щелчке на кнопку, показан в листинге 8.2.

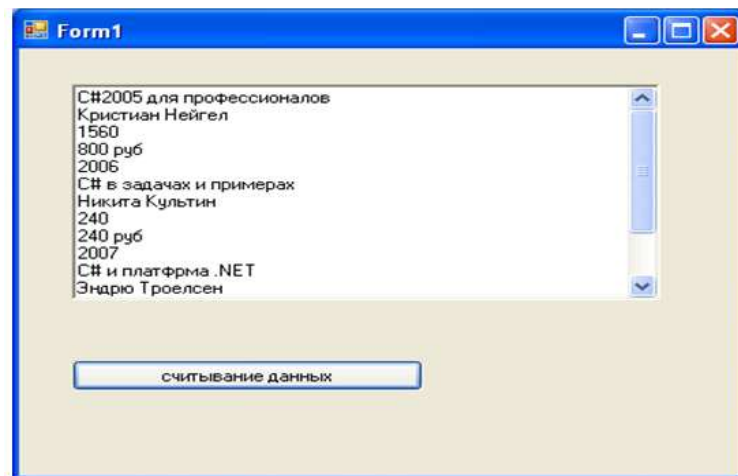


Рис. 8.2. Результаты считывания из **XML**-документа

Листинг 8.2. Код обработчика щелчка по кнопке

```
// Очистка элемента richTextBox1
richTextBox1.Clear();
// Вызов статического метода Create(), возвращающего объект класса
// Файл book.xml находится в том же месте, что и исполняемый файл
// программы
XmlReader rdr = XmlReader.Create("book.xml");
// Метод Read() перемещает на следующий узел XML-документа
while (rdr.Read())
{
    if (rdr.NodeType == XmlNodeType.Text)
        richTextBox1.AppendText(rdr.Value + "\r\n");
}
```

Класс **XmlReader** также может читать данные со строгим контролем типов. Существует несколько методов **ReadElementContentAs**,

выполняющих чтение, среди которых **ReadElementContentAsDecimal()**, **ReadElementContentAsInt()**, **ReadElementContentAsBoolean()** и др.

В листинге 8.3 показано, как считывать значения в десятичном формате и выполнять над ними математические операции. В рассматриваемом случае цена элемента увеличивается на 25 %. Результаты выполнения этого кода показаны на рис. 8.3.

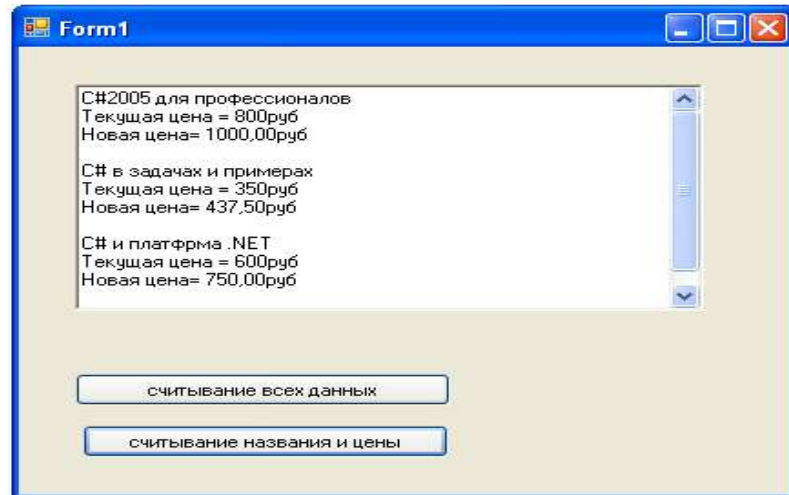


Рис. 8.3. Результаты считывания из XML-документа только названия и цены книг

Листинг 8.3. Чтение данных со строгим контролем типов

```
// Очистка элемента richTextBox1
richTextBox1.Clear();
// Создание потока для чтения из файла book.xml
XmlReader rdr = XmlReader.Create("book.xml");
while (rdr.Read())
{
    if (rdr.NodeType == XmlNodeType.Element)
    {
        // Проверка имени элемента
        if (rdr.Name == "PRICE")
        {
            // Метод ReadElementContentAsDecimal() выполняет
            // преобразование содержимого элемента к типу decimal
            decimal price = rdr.ReadElementContentAsDecimal();
            richTextBox1.AppendText("Текущая цена = " + price +
                "руб\r\n");

            // Изменение цены на 25 %
            price += price * (decimal).25;
            richTextBox1.AppendText("Новая цена = " + price +
                "руб\r\n\r\n");
        }
    }
}
```

```

    }

    else if (rdr.Name == "TITLE")
richTextBox1.AppendText(rdr.ReadElementContentAsString() + "\r\n");
    }
}

```

Класс XmlDocument. Этот класс и производный от него класс **XmlDataDocument** используются в библиотеке **.NET** для представления объектной модели документа **DOM**.

В отличие от класса **XmlReader**, класс **XmlDocument** предлагает возможности не только чтения, но и записи, а также произвольного доступа к дереву **DOM**.

Рассмотрим пример, в котором создается объект класса **XmlDocument**, загружается документ с диска и отображается окно списка с названиями книг (рис. 8.4).

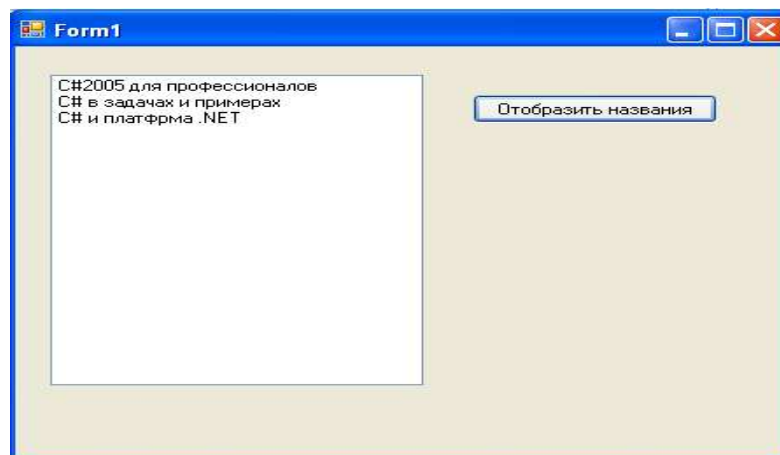


Рис. 8.4. Отображение названий книг в списке.

В классе формы приложения создадим объект класса **XmlDocument**:

```

XmlDocument _doc = new XmlDocument();

```

Код обработчика щелчка по кнопке приведен в листинге 8.4.

Листинг 8.4. Загрузка названий книг в список

```

_doc.Load("book.xml");
// Получить только те узлы, которые нужны
XmlNodeList nodeList = _doc.GetElementsByTagName("TITLE");
// Просмотр в цикле класса XmlNodeList

```

```
foreach (XmlNode node in nodeList)
    listBox1.Items.Add(node.InnerText);
```

Введем в приложение возможность вывода сведений о книге, название которой выделено в списке, для чего добавим обработчик события **listBox1_SelectedIndexChanged** так, как показано в листинге 8.5.

Листинг 8.5. Обработчик щелчка по элементу списка

```
private void listBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // Создание строки поиска
    string srch = "KATALOG/BOOK[TITLE='" + list
Box1.SelectedItem.ToString() + "']";
    // Поиск дополнительных данных
    XmlNode foundNode = _doc.SelectSingleNode(srch);
    if (foundNode != null)
        MessageBox.Show(foundNode.OuterXml);
    else
        MessageBox.Show("Not found");
}
```

Результаты работы приложения показаны на рис. 8.5.

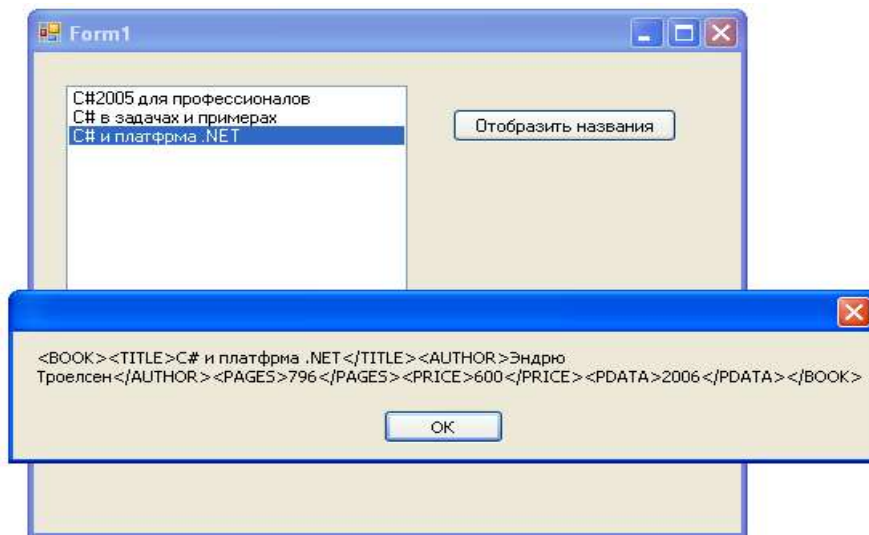


Рис. 8.5. Вывод сведений о выделенном элементе списка

С помощью класса **XmlDocument** также можно вставлять узлы в существующий документ, для чего используется метод **CreateElement()**.

Например, для создания нового элемента **BOOK** необходимо записать следующий код:

```
XmlElement newBook = _doc.CreateElement("BOOK");
```


Создать элементы, вложенные в элемент **BOOK**, можно с помощью следующего кода:

```
// Создание нового элемента AUTOR  
XmlElement newAuthor = _doc.CreateElement("AUTOR");  
newAuthor.InnerText = "С. Байдачный";  
newBook.AppendChild(newAuthor);
```

Полный код обработчика щелчка по кнопке приведен в лис-тинге 8.6, результаты его работы показаны на рис. 8.6.

Листинг 8.6. Обработчик щелчка по кнопке

```
private void button1_Click(object sender, EventArgs e)  
{  
    _doc.Load("book.xml");  
  
    XmlElement newBook = _doc.CreateElement("BOOK");  
    // Создание нового элемента TITLE  
    XmlElement newTitle = _doc.CreateElement("TITLE");  
    newTitle.InnerText = ".NET Framework 2.0";  
    newBook.AppendChild(newTitle);  
  
    // Создание нового элемента AUTOR  
    XmlElement newAuthor = _doc.CreateElement("AUTOR");  
    newAuthor.InnerText = "С. Байдачный";  
    newBook.AppendChild(newAuthor);  
  
    // Создание нового элемента PAGES  
    XmlElement newpages = _doc.CreateElement("PAGES");  
    newpages.InnerText = "498";  
    newBook.AppendChild(newpages);  
  
    // Создание нового элемента PRICE  
    XmlElement newprice = _doc.CreateElement("PRICE");  
    newprice.InnerText = "590";  
    newBook.AppendChild(newprice);  
  
    // Создание нового элемента PDATA  
    XmlElement newpdata = _doc.CreateElement("PDATA");  
    newpdata.InnerText = "2006";  
    newBook.AppendChild(newpdata);  
  
    // Добавление в текущий документ  
    _doc.DocumentElement.AppendChild(newBook);  
    // Запись документа на диск  
    XmlTextWriter tr = new XmlTextWriter("bookEdit.xml", null);  
    tr.Formatting = Formatting.Indented;  
    _doc.WriteContentTo(tr);  
  
    tr.Close();
```

```

// Загрузка listBox1 со всеми названиями книг, включая новое
XmlNodeList nodeList = _doc.GetElementsByTagName("TITLE");
// Просмотр в цикле класса XmlNodeList
foreach (XmlNode node in nodeList)
listBox1.Items.Add(node.InnerText);
}

```

При использовании классов **XmlDocument** и **XmlReader** необходимо учитывать следующие особенности. Если требуется возможность произвольного доступа к документу, то следует применять класс **XmlDocument**, а если нужна потоковая модель, то классы, основанные на классе **XmlReader**. Класс **XmlDocument** отличается большой гибкостью, но его требования к памяти являются более высокими, чем у класса **XmlReader**, а производительность при считывании документа – более низкой.

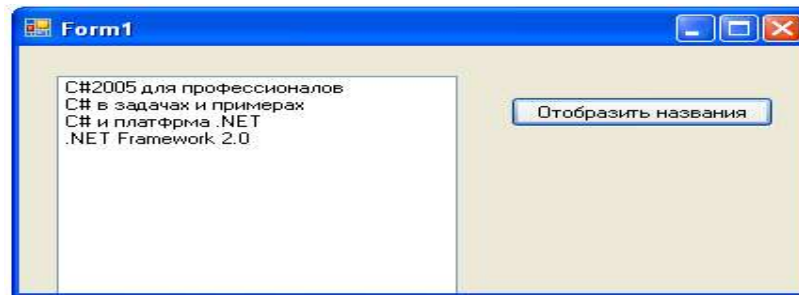


Рис. 8.6. Окно работающего приложения с добавленным узлом

Класс XPathNavigator. Класс **XPathNavigator** является частью пространства имен **System.Xml.XPath**, созданного для увеличения быстродействия. Это пространство обеспечивает только чтение документов, следовательно средства редактирования в нем отсутствуют, а классы построены так, чтобы обеспечить быстрое выполнение на заданном **XML**-документе циклов и операций выбора в стиле курсора.

Наилучшим способом использования классов из пространства имен **System.Xml.XPath** является просмотр документа **book.xml** в цикле.

Разместим на форме элементы **listBox** и **button**. Код обработчика щелчка по кнопке приведен в листинге 8.7, результаты выполнения этого кода – на рис. 8.7.

Листинг 8.7. Обработчик щелчка по кнопке

```

private void button1_Click(object sender, EventArgs e)
{
// Создание объекта с именем doc класса XPathDocument и передача

```

```

// в его конструктор имени XML-файла book.xml
XPathDocument doc = new XPathDocument("book.xml");
// Создание объекта с именем nav класса XPathNavigator на базе объекта
// XPathDocument. Объект nav может использоваться только для чтения
// данных
XPathNavigator nav = ((IXPathNavigable)doc).CreateNavigator();
// Создание объекта XPathNodeIterator для узлов каталога
// и его дочерних узлов
XPathNodeIterator iter = nav.Select("/KATALOG/BOOK");

while (iter.MoveNext())
{
// Метод SelectDescendants( ) класса XPathNavigator выбирает все
// узлы-потомки текущего узла, соответствующие условиям выбора
XPathNodeIterator newIter =
iter.Current.SelectDescendants(XPathNodeType.Element, false);

// Заполнение списка именами и значениями элементов
while (newIter.MoveNext())
listBox1.Items.Add(newIter.Current.Name + ": " +
newIter.Current.Value);
}

// Добавление в список элемента, содержащего значение общей стоимости
// книг
listBox1.Items.Add("=====");
listBox1.Items.Add("Общая стоимость = " +
nav.Evaluate("sum(/KATALOG/BOOK/PRICE)")+"руб");
}

```

Примечание. Если объект класса **XPathNavigator** создать на базе класса **XmlDocument**, то он может быть использован и для редактирования документа.

Разработка приложения КНИГИ. Рассмотрим пример разработки приложения, отображающего информацию о каталоге книг. Хранилищем данных о книгах является файл с именем **book.xml**, содержимое которого было представлено в листинге 8.1. Организуем фильтрацию по всем элементам данных: названию, автору, количеству страниц, цене и дате издания книги.

Form1

TITLE: С#2005 для профессионалов
 AUTHOR: Кристиан Нейгел
 PAGES: 1560
 PRICE: 800
 PDATA: 2006
 TITLE: С# в задачах и примерах
 AUTHOR: Никита Культин
 PAGES: 240
 PRICE: 350
 PDATA: 2007
 TITLE: С# и платформа .NET
 AUTHOR: Эндрю Троелсен
 PAGES: 796
 PRICE: 600
 PDATA: 2006
 TITLE: .NET Framework 2.0
 AUTHOR: С. Байдачный
 PAGES: 498
 PRICE: 590
 PDATA: 2006
 =====
 Общая стоимость = 2340руб

Суммарная стоимость книг

Рис. 8.7. Результаты выполнения кода обработчика щелчка по кнопке **Суммарная стоимость книг**

Для проектирования интерфейса приложения на форме разместим элементы управления, как показано на рис. 8.8.

Form1

0 of (0)

Введите строку для фильтрации

☐ Названию
☐ Автору
☐ Количеству страниц
☐ Цене книги
☒ Дате издания

Результат фильтрации

Отобразить все

Сохранить изменения

bindingNavigator1 bindingSource1

Рис. 8.8. Размещение элементов управления на форме при проектировании приложения **Книги**

Поясним назначение элементов управления, не использовавшихся в предыдущих лабораторных работах.

Элемент **BindingSource** применяется во многих случаях. В частности, он упрощает привязку элементов управления на форме к данным, предоставляя средства управления денежными единицами,

передавая уведомления об изменениях и выполняя другие функции связи между элементами управления **Windows Forms** и источниками данных. Это достигается путем присоединения компонента **BindingSource** к источнику данных с помощью свойства **DataSource**. В сложных сценариях привязки можно дополнительно указать в свойстве **DataMember** определенный столбец или список в источнике данных. Затем выполняется привязка элементов управления к компоненту **BindingSource**. Все последующие взаимодействия с данными осуществляются с помощью вызовов этого компонента.

Элемент управления **BindingNavigator** представляет стандартный способ навигации и управления данными на форме. В большинстве случаев этот элемент управления соединяется попарно с элементом управления **BindingSource**, чтобы обеспечивать перемещение между записями данных на форме и взаимодействовать с ними. В этих случаях свойство **BindingSource** устанавливается на соответствующий компонент **System.Windows.Forms.BindingSource**, который действует в качестве источника данных.

Элемент управления **DataGriedView** отображает данные в настраиваемой сетке.

Код приложения **Книги** представлен в листинге 8.8.

Листинг 8.8. Код приложения Книги

```
using System.IO;
using System.Xml;
using System.Xml.Schema;

namespace testReadDataSet
{
    public partial class Form1 : Form
    {
        // Переменная для имени файла
        string file_name = "book.xml";
        XmlTextReader xml_read;

        // Класс DataSet представляет собой расположенный в памяти кэш данных,
        // загружаемых из источника данных
        // Класс DataSet состоит из коллекции таблиц класса DataTable
        DataSet DataXML;

        // Класс DataTable представляет одну таблицу с данными в памяти
        DataTable MyDatatable = null;

        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
```

```

    {
        DataXML = new DataSet();

// Создание потока для чтения данных из файла, имя которого
// хранится в строке file_name
        FileStream fs = new FileStream(file_name, FileMode.Open);
        xml_read = new XmlTextReader(fs);
        DataXML.ReadXml(xml_read, XmlReadMode.InferSchema);
        MyDatatable = DataXML.Tables[0];
        bindingSource1.DataMember = DataXML.Tables[0].ToString();
        bindingSource1.DataSource = DataXML.Tables[0];

this.bindingNavigator1.BindingSource = bindingSource1;
        dataGridView1.DataSource = bindingSource1;
        dataGridView1.EditMode = DataGridViewEditMode.EditOnEnter;

// Задание размеров и заголовков столбцов элемента dataGridView1
        dataGridView1.Columns[0].Width = 250;
        dataGridView1.Columns[0].HeaderText = "Название";
        dataGridView1.Columns[1].Width = 160;
        dataGridView1.Columns[1].HeaderText = "Автор";
        dataGridView1.Columns[2].Width = 55;
        dataGridView1.Columns[2].HeaderText = "Страниц";
        dataGridView1.Columns[3].Width = 55;
        dataGridView1.Columns[3].HeaderText = "Цена";
        dataGridView1.Columns[4].Width = 55;
        dataGridView1.Columns[4].HeaderText = "Год";
        fs.Close();
    }

// Щелчок по кнопке Сохранить изменения
private void button1_Click(object sender, EventArgs e)
    {
        DataXML.WriteXml(file_name, XmlWriteMode.IgnoreSchema);
    }

// Щелчок по кнопке Результат фильтрации
private void button2_Click(object sender, EventArgs e)
    {
// Если строка для поиска данных является пустой
        if (textBox1.Text == " ")
        {
            MessageBox.Show("Вы не ввели данные для выполнения
            фильтрации в базе данных!");
            return;
        }

        DataView dv = new DataView(DataXML.Tables[0]);
// Если нажата радиокнопка для поиска данных по дате
        if (radioButton4.Checked == true)

```

```

        {
            uint g = Convert.ToUInt32(textBox1.Text);
            dv.RowFilter = "PDATA='" + g + "'";
        }

// Если нажата радиокнопка для поиска данных по названию
if (radioButton1.Checked == true)
{
    string t = textBox1.Text;
    dv.RowFilter = "TITLE='" + t + "'";
}

// Если нажата радиокнопка для поиска данных по автору
if (radioButton5.Checked == true)
{
    string a = textBox1.Text;
    dv.RowFilter = "AUTHOR='" + a + "'";
}

// Если нажата радиокнопка для поиска данных по количеству страниц
if (radioButton2.Checked == true)
{
    uint p = Convert.ToUInt32(textBox1.Text);
    dv.RowFilter = "PAGES='" + p + "'";
}

// Если нажата радиокнопка для поиска данных по цене
if (radioButton3.Checked == true)
{
    uint pr = Convert.ToUInt32(textBox1.Text);
    dv.RowFilter = "PRICE='" + pr + "'";
}
bindingSource1.DataSource = dv;
this.bindingNavigator1.BindingSource = bindingSource1;
dataGridView1.EditMode = false;
dataGridView1.DataSource = bindingSource1;
dataGridView1.EditMode = DataGridViewEditMode.EditProgrammatically;
}

// Щелчок по кнопке Отобразить все
private void button3_Click(object sender, EventArgs e)
{
    bindingSource1.DataSource = DataXML.Tables[0];
    this.bindingNavigator1.BindingSource = bindingSource1;
    dataGridView1.DataSource = bindingSource1;
    dataGridView1.EditMode = DataGridViewEditMode.EditOnEnter;
}
}
}

```

Разработка приложения КАТАЛОГ ТОВАРОВ. Рассмотрим пример разработки приложения, отображающего информацию о каталоге товаров. Хранилищем данных о товарах является файл с именем **catalogue.xml**, содержимое которого представлено в листинге 8.9.

В файле хранятся данные о коде товара, наименовании, цене, фотография товара и его краткое описание. Приложение позволяет просматривать каталог товаров, включая их изображения.

Листинг 8.9. Файл **catalogue.xml**

```
<?xml version="1.0" encoding="utf-8" ?>

<catalogue>
  <unit>
    <uCode>003782</uCode>
    <uTitle>Чайник</uTitle>
    <uMemo>Чайник из нержавеющей стали. Объем: 1,5 л.</uMemo>
    <uPrice>670,50</uPrice>
    <uImage>i001.jpg</uImage>
  </unit>
  <unit>
    <uCode>012345</uCode>
    <uTitle>Чашка с блюдцем</uTitle>
    <uMemo>Чашка с блюдцем. Керамика. Цвет: синий,
      зеленый, красный.</uMemo>
    <uPrice>99,00</uPrice>
    <uImage>i002.jpg</uImage>
  </unit>
  <unit>
    <uCode>01524</uCode>
    <uTitle>Комплект столовой мебели</uTitle>
    <uMemo>Комплект столовой мебели. Стол + 4 стула.
      Материал: массив клена.</uMemo>
    <uPrice>3900,00</uPrice>
    <uImage>i003.jpg</uImage>
  </unit>
  <unit>
    <uCode>01555</uCode>
    <uTitle>Лампа настольная</uTitle>
    <uMemo>Лампа настольная. 40 Вт. Материал:
      керамика.</uMemo>
    <uPrice>549,50</uPrice>
    <uImage>i006.jpg</uImage>
  </unit>
  <unit>
    <uCode>01325</uCode>
    <uTitle>Миска</uTitle>
    <uMemo>Миска керамическая. Цвет: коричневый+
```



```

        бежевый, голубой+белый.</uMemo>
        <uPrice>49,50</uPrice>
        <uImage>i012.jpg</uImage>
    </unit>
    <unit>
        <uCode>01197</uCode>
        <uTitle>Люстра</uTitle>
        <uMemo>Люстра. 40 Вт. Хромированное
        покрытие.</uMemo>
        <uPrice>1549,50</uPrice>
        <uImage>i011.jpg</uImage>
    </unit>
</catalogue>

```

Для проектирования интерфейса приложения на форме следует разместить элементы управления, как показано на рис. 8.9.

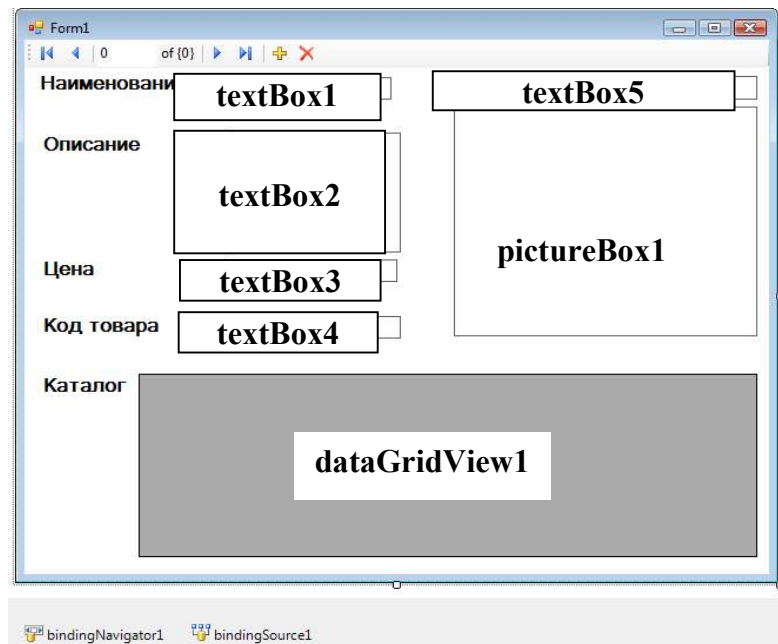


Рис. 8.9. Размещение элементов управления на форме при проектировании приложения **Каталог товаров**
Код приложения представлен в листинге 8.10.

Листинг 8.10. Код приложения **Каталог товаров**

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.IO;
using System.Xml;

```

```

namespace WindowsApplication1
{
    public partial class Form1 : Form
    {
        string file_name = "catalogue.xml";

        XmlTextReader xml_read;
        // Переменная для хранения имени файла

        // Объект DataXML класса DataSet представляет собой расположенный
        // в памяти кэш данных, загружаемых из источника данных
        DataSet DataXML;
        DataTable MyDatatable = null;
        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            DataXML = new DataSet();
            FileStream fs = new FileStream(file_name, FileMode.Open);
            xml_read = new XmlTextReader(fs);
            DataXML.ReadXml(xml_read, XmlReadMode.InferSchema);
            MyDatatable = DataXML.Tables[0];
            bindingSource1.DataMember = DataXML.Tables[0].ToString();
            bindingSource1.DataSource = DataXML.Tables[0];
            this.bindingNavigator1.BindingSource = bindingSource1;
            dataGridView1.DataSource = bindingSource1;
            dataGridView1.EditMode = DataGridViewEditMode.EditOnEnter;
            dataGridView1.Columns[0].Width = 40;
            dataGridView1.Columns[0].HeaderText = "Код";
            dataGridView1.Columns[1].Width = 100;
            dataGridView1.Columns[1].HeaderText = "Наименование";
            dataGridView1.Columns[2].Width = 250;
            dataGridView1.Columns[2].HeaderText = "Описание";
            dataGridView1.Columns[3].Width = 55;
            dataGridView1.Columns[3].HeaderText = "Цена";
            fs.Close();
        }

        // Щелчок в ячейке dataGridView1
        private void dataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
        {
            textBox2.Text =

```

```

        DataXML.Tables[0].Rows[e.RowIndex][2].ToString();
        textBox1.Text
        RowIndex][1].ToString();
        textBox3.Text =
        DataXML.Tables[0].Rows[e.RowIndex][3].ToString();
        textBox4.Text =
        DataXML.Tables[0].Rows[e.RowIndex][0].ToString();
        textBox5.Text =
        DataXML.Tables[0].Rows[e.RowIndex][4].ToString();
        pictureBox1.Image = new Bitmap(textBox5.Text);    }

// Щелчок в левом поле строки dataGridView1
private void dataGridView1_RowHeaderMouseClick(object sender,
DataGridViewCellEventArgs e)
{
    textBox2.Text =
    DataXML.Tables[0].Rows[e.RowIndex][2].ToString();
    textBox1.Text =
    DataXML.Tables[0].Rows[e.RowIndex][1].ToString();
    textBox3.Text =
    DataXML.Tables[0].Rows[e.RowIndex][3].ToString();
    textBox4.Text =
    DataXML.Tables[0].Rows[e.RowIndex][0].ToString();
    textBox5.Text =
    DataXML.Tables[0].Rows[e.RowIndex][4].ToString();
    pictureBox1.Image = new Bitmap(textBox5.Text);
}
}
}
}

```

Порядок выполнения работы

1. Изучить теоретические сведения и примеры, представленные ниже.
2. Ответить на контрольные вопросы.
3. Разработать **Windows**-приложение в соответствии с вариантом практического задания.
4. Для созданного приложения разработать **UML**-диаграмму классов.
5. Составить отчет в электронном виде, который должен содержать титульный лист, цель лабораторной работы, задание, ответы на контрольные вопросы, листинг программы и результаты ее работы.

Контрольные вопросы и задания

1. Расшифруйте аббревиатуру XML.
2. Для каких целей может использоваться язык XML?
3. Дайте характеристику XML-документа.
4. Объясните значение аббревиатуры DOM.
5. В каком пространстве имен определены классы библиотеки **FCL**, поддерживающие работу с XML-документами?
6. Каким образом можно выполнить создание XML-документа в среде **Visual Studio** платформы .NET?
7. Сколько корневых элементов может содержать XML-документ?
8. Каким образом маркируются данные в XML-документе?
9. Перечислите правила корректного формирования XML-документа.
10. Назовите классы библиотеки **FCL**, позволяющие выполнить чтение XML-файлов.
11. Какие различия имеются в использовании классов **XmlReader** и **XmlDocument**?
12. Какой класс предоставляет возможности для редактирования XML-документа?
13. Каким образом можно выполнить чтение данных из XML-документа со строгим контролем типов?
14. В каких случаях следует использовать класс **XPathNavigator**?
15. Как программно выполнить вставку узла в XML-документ?
16. Каким образом можно выполнить чтение данных из XML-документа со строгим контролем типов данных?
17. С помощью какого элемента можно отобразить данные на форме в виде таблицы?

Варианты практических заданий

1. Разработать приложение **Телефонный справочник**.
Разработать структуру XML-файла для хранения фамилии, имени, отчества абонентов, их номеров телефонов и адресов. Предусмотреть возможность поиска данных по фамилии и номеру телефона.
2. Разработать приложение **Автосалон**.
Разработать структуру XML-файла для хранения таких данных, как марка автомобиля, страна производителя, цена, его изображение. Предусмотреть возможность поиска данных по марке автомобиля и его стоимости.
3. Разработать приложение **Мой фотоальбом**.
Разработать структуру XML-файла для хранения фотографии, ее названия, даты съемки. Предусмотреть возможность поиска данных по названию фотографии и дате.
4. Разработать приложение **Книги**.

Разработать структуру XML-файла для хранения таких данных, как название, автор, жанр, год, количество страниц, тираж, отпечатанные листы. Предусмотреть возможность сортировки данных по автору книги и поиска данных по названию и жанру.

5. Разработать приложение **Транспорт**.

Разработать структуру XML-файла для хранения таких данных, как наименование транспорта, тип, год выпуска, максимальная скорость, объем двигателя, расход, объем бензобака. Предусмотреть возможность сортировки данных по году выпуска и поиска данных по наименованию и году выпуска.

6. Разработать приложение **Продовольственные товары**.

Разработать структуру XML-файла для хранения таких данных, как наименование товара, дата выпуска, срок хранения, цена. Предусмотреть возможность сортировки данных по наименованию товара и поиска данных по наименованию и цене.

7. Разработать приложение **Студенты**.

Разработать структуру XML-файла для хранения таких данных, как фамилия, имя, отчество, год поступления, курс, наименование группы, средний балл. Предусмотреть возможность сортировки данных по наименованию группы и поиска данных по фамилии и среднему баллу.

8. Разработать приложение **Объекты недвижимости**.

Разработать структуру XML-файла для хранения таких данных, как адрес, количество комнат, этажность, количество квартир на этаж, количество подъездов, всего квартир. Предусмотреть возможность сортировки данных по количеству комнат и поиска данных по адресу и количеству комнат.

9. Разработать приложение **Спортсмены**.

Разработать структуру XML-файла для хранения таких данных, как фамилия, имя, отчество, вид спорта, разряд, фотография. Предусмотреть возможность сортировки данных по фамилии и поиска данных по виду спорта.

10. Разработать приложение **Периодические издания**.

Разработать структуру XML-файла для хранения таких данных, как название, тип, количество страниц, частота выпуска, тираж, количество выпусков в год. Предусмотреть возможность сортировки данных по наименованию и поиска данных по году выпуска.

11. Разработать приложение **Отдел кадров**.

Разработать структуру XML-файла для хранения таких данных, как фамилия, имя, отчество, отдел, должность, дата приема на работу, стаж работы. Предусмотреть возможность сортировки данных по фамилии и поиска данных по фамилии и должности.

12. Разработать приложение **Научно-исследовательские разработки.**

Разработать структуру XML-файла для хранения таких данных, как наименование, дата начала, дата завершения, срок работы, область исследования, количество сотрудников, фамилии, имена, отчества сотрудников. Предусмотреть возможность сортировки данных по наименованию и поиска данных по области исследования и дате завершения.

13. Разработать приложение **Программное обеспечение.**

Разработать структуру XML-файла для хранения таких данных, как наименование, назначение, количество дисков, объем после установки (полной, минимальной, типичной), цена. Предусмотреть возможность сортировки данных по наименованию и поиска данных по назначению и цене.

14. Разработать приложение **Комплектующие ЭВМ.**

Разработать структуру XML-файла для хранения таких данных, как наименование, модель, частота, объем памяти, стоимость, количество комплектующих. Предусмотреть возможность сортировки данных по наименованию и поиска данных по модели и цене.

15. Разработать приложение **Перевозки.**

Разработать структуру XML-файла для хранения таких данных, как пункт назначения, количество транспорта, дата выезда, время в дороге. Предусмотреть возможность сортировки данных по времени в дороге и поиска данных по пункту назначения и дате выезда.

16. Разработать приложение **Аудиостудия.**

Разработать структуру XML-файла для хранения таких данных, как название группы, количество человек в группе, количество альбомов, фамилия солиста. Предусмотреть возможность сортировки данных по количеству альбомов и поиска данных по названию группы и фамилии солиста.

17. Разработать приложение **Мобильные телефоны.**

Разработать структуру XML-файла для хранения таких данных, как наименование, фирма, стандарт связи, заряд аккумулятора, потребление при ожидании, потребление при разговоре, цена. Предусмотреть возможность сортировки данных по цене и поиска данных по наименованию и цене.

18. Разработать приложение **Учебный план.**

Разработать структуру XML-файла для хранения таких данных, как наименование, название направления подготовки, название дисциплины, отчетность по дисциплине (зачет, экзамен), номер семестра, в котором изучается дисциплина, количество часов.

Предусмотреть возможность сортировки данных по номеру семестра и поиска данных по названию дисциплины и номеру семестра.

19. Разработать приложение **Учебная группа**.

Разработать структуру XML-файла для хранения таких данных, как наименование группы, название направления подготовки, фамилии, имена, отчества студентов и средний балл студента. Предусмотреть возможность сортировки данных по среднему баллу студента и поиска данных по наименованию группы и среднему баллу.

20. Разработать приложение **Кафедра**.

Разработать структуру XML-файла для хранения таких данных, как наименование кафедры, фамилии, имена, отчества преподавателей кафедры, должности преподавателей, наименование дисциплин на кафедре, количество сотрудников. Предусмотреть возможность сортировки данных по количеству сотрудников кафедры и поиска данных по должности преподавателя.

21. Разработать приложение **Расписание занятий**.

Разработать структуру XML-файла для хранения таких данных, как название дня недели, наименование дисциплины, аудитория, время занятий. Предусмотреть возможность сортировки данных по названию дисциплины и поиска данных по названию дня недели.